CloudGuard + GRC Platform Merge

This repository now contains the latest CloudGuard scanner portal plus the GRC platform module from the handoff repository.

Directory Layout

```
Cloud_Vulnerabilities/
  backend/                  # CloudGuard FastAPI API, scan queue, evidence APIs, sandbo
  frontend/                 # CloudGuard scanner portal pages
  deploy/                   # CloudGuard deployment files for Dokploy, E2E, and Digital
  db/init/                  # CloudGuard PostgreSQL schema and migrations
  grc-platform/             # Full GRC platform module merged from the handoff reposito
  docs/                     # CloudGuard architecture and deployment documentation
```

What Runs What

CloudGuard is responsible for scanning and evidence collection:

- AWS, GCP, Kubernetes, IaC, and web scan jobs
- Sandbox demo labs that create vulnerable resources, scan them, then clean them up
- Evidence ingestion through `/api/evidence`
- Compliance summary through `/api/compliance/*`
- Reason-and-Act workflows for validation, remediation approval, and threat modeling

The GRC platform is responsible for governance workflows:

- Compliance program management
- Risk tracking and prioritization
- Evidence review and approval
- Audit workflows
- CERT-In MSME-style evidence mapping
- CloudGuard connector visualization
- MCP evidence normalization through `normalize_cloudguard_evidence`

Evidence Flow

```
CloudGuard scanners / sandbox labs / manual evidence / external scanner outputs
        ↓
CloudGuard evidence ingestion or GRC evidence MCP normalizer
        ↓
Normalized finding and evidence records
        ↓
Control mapping, risk priority, audit status, and remediation tracking
        ↓
GRC dashboard and compliance reports
```

App Entry Point

CloudGuard now exposes a `/grc` route and a navigation item named `GRC`.

- If `GRC_PLATFORM_URL` is set, `/grc` redirects to that hosted GRC dashboard.
- If `GRC_PLATFORM_URL` is not set, `/grc` shows a local bridge page explaining how to run the merged module.

Example:

```
GRC_PLATFORM_URL=https://grc.example.com
```

For early deployments, use subdomain routing:

```
scanner.example.com  -> CloudGuard scanner portal
grc.example.com      -> GRC platform
```

This is simpler than path-based routing because the GRC frontend, Keycloak, API gateway, and service URLs do not need to be rebuilt around a `/grc` base path.

Live Connector Configuration

The GRC browser does not call CloudGuard directly and never receives the connector secret. The GRC controls service calls CloudGuard's protected `/api/connectors/grc/dashboard` endpoint and returns the authorized result to the signed-in GRC user.

Set these variables on the GRC controls service:

```
CLOUDGUARD_API_URL=https://scanner.example.com
CLOUDGUARD_CONNECTOR_CREDENTIALS_JSON={"tenant-id":{"connector_id":"grc-tenant-id","toke
```

The JSON key, `tenant_id`, and the Keycloak user's `organization_id` claim must be the same value. The GRC controls service selects only the credential for the signed-in user's tenant; there is no global connector token or user-header fallback.

Set this on the CloudGuard service so its GRC navigation opens the deployed dashboard:

```
GRC_PLATFORM_URL=https://grc.example.com
```

The CloudGuard GRC page now displays only live connector data. If the connector is unavailable or has no evidence, it shows an explicit empty/error state instead of old demo findings.

Local Development

Run CloudGuard from the repository root:

```
docker compose --profile dev up --build
```

Run the GRC platform separately:

```
cd grc-platform
./start.sh
```

The GRC app normally opens on:

`http://localhost:3000`

CloudGuard normally opens on:

`http://localhost:8000`

Test Labs

The vulnerable lab assets from the handoff repo are available at:

`grc-platform/cloudguard-test-labs/`

Use them only inside isolated sandbox accounts, projects, or clusters. The preferred demo model is not to create and close whole cloud accounts on every click. Instead:

1. Keep a disposable AWS account, GCP project, and Kubernetes cluster ready.
2. Create temporary vulnerable resources inside that isolated environment.
3. Run CloudGuard scans.
4. Ingest findings and proof as GRC evidence.
5. Destroy the temporary resources after scan completion or a short TTL.

Deployment Requirement

The connector code is live-data-backed. Before deploying, configure tenant-specific connector credentials, the CloudGuard API URL, the Keycloak realm clients, and the database migrations described in keycloak-multitenant-deployment.md. The GRC service containers must also be deployed before the GRC application can be used in production.

The live connector is read-only: CloudGuard remains the evidence source of record, while GRC users review the resulting compliance and risk view. Copying CloudGuard artifacts into the GRC object store can be added later if an auditor requires a second retained copy.

Do not overwrite the latest CloudGuard deployment files with the older handoff CloudGuard copy. The latest scanner deployment remains in this repository root, especially the Dokploy `production-lite` stack and sandbox lab worker.